

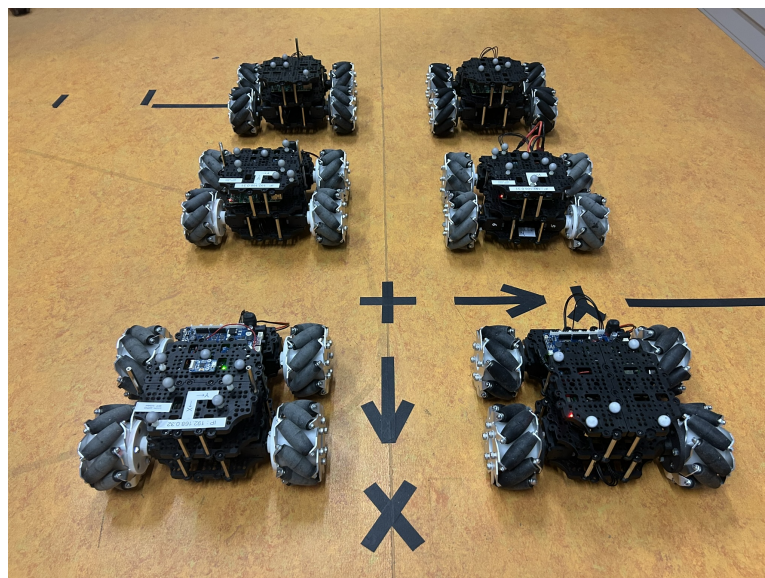
Thibaut Waechter

Promotion : 2024-2025

Année universitaire : 2024-2025

Rapport de Projet de fin d'étude

eSWARM



Organisme d'accueil : iCube

300 Boulevard Sébastien Brandt

67400 Illkirch

Sylvain Durand

sylvain.durand@insa-strasbourg.fr

03/03/25 – 05/09/25



Résumés

Résumé en français

Durant mon stage de fin d'étude au laboratoire ICube de Strasbourg (67).

English summary

During my end-of-study internship at the ICube laboratory in Strasbourg (67).

Remerciements

J'exprime toute ma gratitude au laboratoire ICube et à l'INSA Strasbourg de m'avoir accueilli pour ce stage de recherche, qui a été une expérience formatrice et enrichissante.

Je tiens à remercier tout particulièrement M. Sylvain Durand, mon tuteur de stage, pour son encadrement et ses conseils. Son expertise et sa patience m'ont permis de développer mes compétences en recherche et d'acquérir une plus grande autonomie dans mon travail.

Mes remerciements s'adressent également à M. Renaud Kiefer, M. Thomas Pavot et Mme Maya Pivert pour leur aide précieuse et leurs conseils techniques qui ont contribué à la réussite de ce projet.

Je souhaite aussi exprimer ma reconnaissance à mes camarades Lounès Youbi et Aurélien Le Bastard pour leur soutien et la bonne entente qui a régné tout au long de ce stage.

Ce stage de type recherche m'a beaucoup appris, tant sur le plan technique que personnel. Grâce à lui, j'ai pu développer mon autonomie, découvrir le milieu de la recherche scientifique et d'acquérir une méthodologie de travail rigoureuse qui me sera sans aucun doute précieuse.

Table des matières

1	Introduction	1
2	But du travail	2
3	Contexte	3
4	Etat de l'art	4
5	Descriptions des méthodes utilisées	5
5.1	Utilisations de robots holonomes terrestres	5
5.2	Consensus pour le pilotage de l'essaim	5
5.3	Commande événementielle	5
5.3.1	Commande événementielle classique	5
5.3.2	Commande événementielle prédictive	6
5.4	Autres contributions à [5]	7
5.4.1	Calculs des points cibles	7
5.4.2	Adaptation du coefficient c_1^γ	7
6	Description des réalisations	10
6.1	Outils utilisés	10
6.1.1	Robot Operating System 2 (ROS2) [6]	10
6.1.2	Système Optitrack - Logiciel Motive [7]	10
6.1.3	Turtlebot Mecanum	10
6.1.4	Logiciels utilisés	11
6.2	Essaim de robots	11
6.3	Organisation du système avec Robot Operating System (ROS)	11
7	Présentations des résultats	12
7.1	Algorithme classique	12
7.2	Commande événementielle	14
7.2.1	Commande événementielle classique	14
7.2.2	Commande événementielle prédictive	16
7.3	Comparaisons des différentes méthodes	17
8	Conclusion	18
	Annexes	21
A	Théorie des graphes	22
A.1	Introduction	22
A.2	Définition mathématique	22
A.2.1	Principe	22
A.2.2	Propriétés	23
A.3	Lien avec un essaim de robot	25

B	Contrôle de l'essaim	26
B.1	Introduction	26
B.2	Définition du consensus	26
B.3	Application à un essaim	27
C	Modèle Mecanum	28

Liste des acronymes

IDE Integrated Development Environment. 11

IRIV-AR Imagerie, Robotique et Ingénierie du Vivant - Automatique Robotique. 1

MRR Modular Reconfigurable Robot. 2

RDH Robotique, Data Science et Healthcare technologies. 1

ROS1 Robot Operating System version 1. 27

ROS2 Robot Operating System version 2. 27

S-IoT Système Embarqué - Internet of Things. 1

VRPN Virtual-Reality Peripheral Network. 10

VS Code Visual Studio Code. 11

Table des figures

2.1	Système ModQuad [4].	2
5.1	Illustration du calcul du point cible individuel à partir du barycentre et du vecteur relatif initial.	7
6.1	TurtleBot Mecanum	10
7.1	Graphe de la formation	12
7.2	Trajectoire du barycentre de l'essaim	12
7.3	Trajectoire de chaque robot de l'essaim	12
7.4	Evolution des inter-distances entre les robots	13
7.5	Trajectoire des robots avec une perturbation	13
7.6	Evolution des inter-distances entre les robots avec perturbation	13
7.7	Trajectoire du barycentre de l'essaim	14
7.8	Trajectoire de chaque robot de l'essaim	14
7.9	Evolution des inter-distances entre les robots	15
7.10	Trajectoire des robots avec une perturbation	15
7.11	Evolution des inter-distances entre les robots avec perturbation	15
7.12	Trajectoire du barycentre de l'essaim	16
7.13	Trajectoire de chaque robot de l'essaim	16
7.14	Evolution des inter-distances entre les robots	16
7.15	Trajectoire des robots avec une perturbation	17
7.16	Evolution des inter-distances entre les robots avec perturbation	17
A.1	Exemple de graphe	22
A.2	Graphe orienté	23
A.3	Deux exemples de graphes non connexes	24
A.4	Exemple d'un arbre couvrant (en bleu)	24

1 Introduction

Dans le cadre de mon stage de fin d'étude en Génie Electrique spécialité Système Embarqué - Internet of Things (S-IoT) à l'INSA Strasbourg, j'ai eu l'opportunité d'intégrer l'équipe Robotique, Data Science et Healthcare technologies (RDH) du laboratoire iCube de Strasbourg (67000). Ce stage s'inscrit également dans le cadre de mon double diplôme - Master Imagerie, Robotique et Ingénierie du Vivant - Automatique Robotique (IRIV-AR).

Ma mission a été de développer un système de contrôle distribué pour un essaim de robots terrestres holonomes, appelés robots Mecanum, et optimiser la commande.

Le plan du mémoire est le suivant :

2 But du travail

Ce stage s'inscrit dans la continuité du projet eSWARM et poursuit les travaux d'Augustin Bonnel [1] et de Sylvain Boegeat, deux anciens étudiants de l'INSA. L'objectif de ce projet est de développer un système de drones quadricoptères modulaire et reconfigurable, capable de s'assembler en vol afin d'accomplir des missions qu'un drone seul ne pourrait réaliser, comme par exemple le transport de charges lourdes.

Le projet eSWARM est basé sur des essaims de robots dits reconfigurables et modulaires. Une définition de ce type de système est donnée dans [2] : "Les systèmes de robots modulaires reconfigurables (Modular Reconfigurable Robot (MRR) pour *Modular Reconfigurable Robot*) sont constitués de nombreux modules (ou unités) répétés qui peuvent être réorganisés dans différentes configurations en fonction de la tâche que le robot doit accomplir." Une définition similaire est fournie dans [3].

Il est essentiel d'expliquer les deux termes : *modulaire* et *reconfigurable*. Le premier désigne une architecture composée de modules pouvant s'assembler de diverses manières. Le second caractérise la capacité du système à modifier sa structure au cours de son fonctionnement.

L'une des principale source d'inspiration pour ce projet est le projet ModQuad [4], qui est un système d'essaim modulaire reconfigurable de drones quadricoptères. Il est composé de modules qui peuvent s'assembler pour former des structures plus grandes et plus complexes. Un exemple est donné ci-dessous :



FIGURE 2.1 – Système ModQuad [4].

Pour le stage, il a été décidé de remplacer les drones par des robots terrestres holonomes. Bien que le projet eSWARM soit initialement axé sur les drones aériens, leur utilisation implique des contraintes techniques : autonomie réduite (souvent inférieure à dix minutes) et les risques de crash et de casse de matériel ne sont pas négligeables. Cela peut ralentir le déroulement des expérimentations.

L'un des objectifs de ce stage est aussi l'optimisation des lois de commande dans ce contexte d'essaim de robot. Il s'agit de concevoir une commande distribuée permettant de piloter efficacement la formation de l'essaim, tout en assurant sa stabilité, sa réactivité et sa robustesse.

3 Contexte

4 Etat de l'art

5 Descriptions des méthodes utilisées

5.1 Utilisations de robots holonomes terrestres

Lister avantages et inconvénients par rapport aux drones

5.2 Consensus pour le pilotage de l'essaim

L'algorithme retenu pour ce stage est celui de [5], qui est un algorithme de consensus distribué. Il s'inspire grandement de **ARTICLEARETROUVER**, qui est utilisé pour piloter un essaim de drones, ce qui permet de rester dans la continuité du projet eSWARM. De plus, l'algorithme est relativement simple à mettre en place, et est basé sur des PIDs, ce qui est un avantage pour la compréhension et la mise en œuvre.

- Simplicité
- Adaptable aux drones
- Utilisations de PIDs

5.3 Commande événementielle

5.3.1 Commande événementielle classique

La commande événementielle mise en place consiste à calculer et envoyer une nouvelle commande aux moteurs uniquement lorsque certaines conditions sont réunies, afin de réduire la fréquence des mises à jour et donc la charge de calcul. Contrairement à une commande classique où chaque robot reçoit une consigne à chaque cycle, ici la mise à jour n'est déclenchée que si un événement est détecté. Comme dans le papier de [5], il existe deux commandes : u_i^α pour maintenir la formation et u_i^γ pour l'attraction vers le point cible, il faut au moins une condition par commande. Les conditions sont les suivantes :

- **Écart de distance avec les voisins** : Pour chaque voisin j de i , si l'écart entre la distance actuelle d_{ij} et la distance désirée d_{ij}^* dépasse un seuil δ_d , alors une commande est calculée :

$$|d_{ij} - d_{ij}^*| > \delta_d$$

- **Changement du point cible global** : Si la position du point cible change, tous les robots mettent à jour leur commande :

$$\mathbf{g}^{\text{new}} \neq \mathbf{g}^{\text{old}}$$

- **Proximité ou éloignement significatif de la cible** : Si la distance entre le robot i et sa cible devient inférieure à δ_c , ou si cette distance varie de plus de δ_t par rapport à la précédente, une commande est envoyée :

$$\|\mathbf{p}_i - \mathbf{p}_i^*\| < \delta_c$$

ou $\|\mathbf{p}_i - \mathbf{p}_i^*\| - \|\mathbf{p}_i^{\text{old}} - \mathbf{p}_i^{*, \text{old}}\| > \delta_t$

Les seuils utilisés sont notés :

- δ_d : seuil d'écart de distance avec les voisins
- δ_t : seuil de changement de position cible individuelle
- δ_c : seuil de distance à la cible individuelle

Symbole	Description	Valeur numérique
δ_d	Écart de distance avec voisins	0,05 m
δ_t	Changement de cible individuelle	0,02 m
δ_c	Proximité à la cible individuelle	0,1 m

TABLE 5.1 – Seuils utilisés pour la commande événementielle

5.3.2 Commande événementielle prédictive

La commande événementielle prédictive a pour objectif de limiter les échanges d'informations entre les robots. Ici, chaque robot a besoin de la position de ses voisins pour calculer sa commande. Ainsi, la logique de la commande événementielle prédictive est la suivante : chaque robot prédit la position de ses voisins à partir de leur dernière position publiée et de leur vitesse. Une nouvelle position n'est publiée que si l'erreur de prédiction dépasse un certain seuil.

Chaque robot utilise la même méthode de prédiction pour ses voisins : il estime leur position actuelle en ajoutant à leur dernière position publiée le produit de leur vitesse et du temps écoulé. De plus, chaque robot connaît sa propre position réelle (mesurée) et sa position prédite (calculée comme pour ses voisins). Il peut donc comparer ces deux valeurs et déterminer précisément quand la prédiction devient trop imprécise, ce qui déclenche la publication de sa vraie position.

Note : Chaque robot publie désormais à la fois sa position et sa vitesse. Chacun doit disposer des mêmes informations pour effectuer les mêmes calculs de prédiction. Ce choix implique que, lors de chaque publication, la quantité d'information échangée est doublée (position et vitesse), mais l'objectif est de réduire fortement la fréquence de ces publications, ce qui permet théoriquement de diminuer la charge globale de communication. La formule de la prédiction de position pour le robot i est :

$$\hat{\mathbf{p}}_i = \mathbf{p}_i^{\text{pub}} + \mathbf{v}_i \cdot (t_{\text{actuel}} - t_{\text{pub}})$$

où $\mathbf{p}_i^{\text{pub}}$ est la dernière position publiée, $\mathbf{v}_i$ est la dernière vitesse publiée par le robot (et non une vitesse estimée), et $t_{\text{actuel}} - t_{\text{pub}}$ le temps écoulé depuis la dernière publication.

La condition de publication d'une nouvelle position pour le robot i est la suivante : si l'écart entre la position réelle $\mathbf{p}_i$ et la position estimée $\hat{\mathbf{p}}_i$ (calculée à partir de la dernière position publiée et de la vitesse) dépasse un seuil δ_p , alors le robot publie sa nouvelle position :

$$\|\mathbf{p}_i - \hat{\mathbf{p}}_i\| > \delta_p$$

Avec δ_p le seuil d'erreur de prédiction de position

Symbole	Description	Valeur numérique
δ_p	Erreur de prédiction de position	0,05 m

TABLE 5.2 – Seuil utilisé pour la commande prédictive

5.4 Autres contributions à [5]

5.4.1 Calculs des points cibles

Pour piloter l'essaim, un unique point cible global est envoyé à l'ensemble des robots de l'essaim. Cependant, chaque robot ne cherche pas à atteindre ce point directement. Au lieu de cela, chacun calcule son propre point cible individuel en réalisant la translation du point-cible globale par un **vecteur relatif** correspondant à sa position initiale par rapport au barycentre de l'essaim. Formellement, pour le robot i , le point cible individuel $\mathbf{p}_r^i$ est donné par :

$$\mathbf{p}_r^i = \mathbf{p}_{\text{goal}} + (\mathbf{p}_i^{\text{init}} - \mathbf{p}_{\text{bary}}^{\text{init}}) \quad (5.1)$$

où $\mathbf{p}_{\text{goal}}$ est la cible globale, $\mathbf{p}_i^{\text{init}}$ la position initiale du robot i , et $\mathbf{p}_{\text{bary}}^{\text{init}}$ le barycentre initial de l'essaim.

L'intérêt de cette approche est de préserver la formation relative des robots à l'arrivée l'essaim. Ainsi, même si tous les robots reçoivent la même consigne globale, chacun vise une position différente qui maintient la structure de l'essaim autour du barycentre.

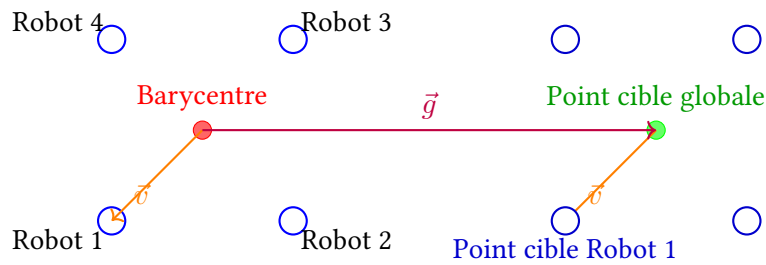


FIGURE 5.1 – Illustration du calcul du point cible individuel à partir du barycentre et du vecteur relatif initial.

La Figure 5.1 ci-dessus illustre la méthode. Le vecteur relatif $\vec{v}$ permet de déterminer la cible individuelle du robot. Le vecteur $\vec{g}$ relie le barycentre au point cible global et symbolise le déplacement global de l'essaim.

5.4.2 Adaptation du coefficient c_1^γ

Comme expliqué précédemment, le coefficient c_1^γ détermine la force avec laquelle chaque robot est attiré vers son point cible. Une valeur de c_1^γ trop élevée permet à l'essaim de converger rapidement vers la cible ; cela augmente la vitesse de déplacement des robots vers leurs points. Cependant, en présence de perturbations (par exemple, un robot décalé ou une erreur de formation), maintenir une attraction vers le point cible peut aggraver la situation : les robots continueraient à avancer vers la cible sans corriger la formation car le terme u_i^α , responsable du maintien de la formation, met un certain temps à converger pour supprimer l'erreur. Pour éviter cela, l'idée est d'adapter dynamiquement c_1^γ : lorsqu'une perturbation est détectée (écart de distance par exemple), c_1^γ est réduit afin de ralentir, voire stopper, le déplacement global de l'essaim. Cela laisse le temps au système de corriger la formation avant de poursuivre la progression vers la cible.

Pour cela, les ratios de distance et des erreurs d'angle entre robots sont continuellement calculés et comparés à des seuils. Si les tolérances sont respectées, c_1^γ garde sa valeur normale, et l'essaim se déplace normalement. Sinon, il est réduit selon la gravité de la perturbation. Le coefficient c_1^γ est adapté selon trois niveaux de fonctionnement :

1. **Cas extrêmes** : arrêt quasi-total ($c_1^\gamma \approx 0$)
2. **Formation correcte** : valeur nominale ($c_1^\gamma = 0.2$)
3. **Formation imparfaite** : réduction progressive

Avant tout calcul, le système vérifie si les ratios de distance (pour toutes les arêtes du graphe) sont dans des plages critiques :

$$r_{\min} = \min_j \left(\frac{d_{ij}^{\text{actuelle}}}{d_{ij}^{\text{désirée}}} \right) < 0.3 \quad \text{ou} \quad r_{\max} = \max_j \left(\frac{d_{ij}^{\text{actuelle}}}{d_{ij}^{\text{désirée}}} \right) > 2.2 \quad (5.2)$$

Dans ces cas critiques, $c_1^\gamma = 0$ pour éviter les collisions ou la perte de formation. 0.3 et 2.2 sont des seuils empiriques basés sur les tests effectués.

La valeur nominale n'est appliquée que si **toutes** les conditions suivantes sont simultanément respectées :

Condition sur les distances :

$$(1 - \tau_d) \leq r_{\min} \quad \text{et} \quad r_{\max} \leq (1 + \tau_d) \quad (5.3)$$

Condition sur les angles :

$$|\theta_{\text{erreur},j}| \leq \tau_\theta \quad \forall j \in \mathcal{N}_i \quad (5.4)$$

où τ_d et τ_θ sont des tolérances sur les distances et angles respectivement. L'erreur angulaire est calculée par rapport à l'angle de formation cible. Si les conditions pour la valeur nominale ne sont pas respectées, le coefficient est réduit selon :

$$c_1^\gamma = c_{1,\text{base}}^\gamma \cdot \min(f_d, f_\theta) \quad (5.5)$$

Les facteurs de réduction f_d et f_θ sont calculés comme :

$$f_d = f_{\text{proximité}} \cdot f_{\text{séparation}} \cdot f_{d,\text{add}} \quad (5.6)$$

$$f_\theta = f_{\text{erreur angle}} \cdot f_{\theta,\text{add}} \quad (5.7)$$

Les sous-facteurs utilisent des réductions exponentielles :

$$f_{\text{proximité}} = \begin{cases} e^{-k_{\min} \cdot \delta_{\min}} & \text{si } r_{\min} < s_{\min} \\ 1 & \text{sinon} \end{cases} \quad (5.8)$$

où $\delta_{\min} = \frac{s_{\min} - r_{\min}}{s_{\min}}$ est la déviation normalisée et $s_{\min}$, $s_{\max}$ sont des seuils de tolérance.

$$f_{\text{séparation}} = \begin{cases} e^{-k_{\max} \cdot \delta_{\max}} & \text{si } r_{\max} > s_{\max} \\ 1 & \text{sinon} \end{cases} \quad (5.9)$$

Des facteurs additionnels $f_{d,\text{add}}$ et $f_{\theta,\text{add}}$ appliquent des réductions supplémentaires lorsque les seuils sont dépassés.

Paramètre	Mode normal
τ_d	$\pm 15\%$
τ_θ	$\pm 15^\circ$
$s_{\min}, s_{\max}$	0.90, 1.10
$k_{\min}, k_{\max}$	0.4
k_θ	0.2
$f_{d,\text{add}}, f_{\theta,\text{add}}$	0.6

TABLE 5.3 – Paramètres d'implémentation selon le mode

Chaque robot calcule donc indépendamment sa propre valeur de c_1^γ en fonction de ses mesures locales. Cela n'est pas problématique pour la cohésion de l'essaim : si un robot détecte une perturbation et réduit son c_1^γ , il ralentit son déplacement vers le point cible. Les autres robots, via les tolérances sur les distances et les angles, vont également détecter ce ralentissement (car leurs propres ratios vont sortir des seuils) et adapter leur c_1^γ en conséquence. Ainsi, même si les valeurs de c_1^γ diffèrent temporairement, le terme u_i^α (responsable du maintien de la formation) reprend naturellement le dessus et assure la cohésion et la correction de la formation avant la reprise du déplacement global.

6 Description des réalisations

6.1 Outils utilisés

6.1.1 Robot Operating System 2 (ROS2) [6]

Robot operating system, ou ROS est un environnement virtuel conçu pour le développement d'applications robotiques. Il est accompagné d'un ensemble d'outils et bibliothèques servant à développer des systèmes robotiques, comme par exemple une ToolBox Matlab/Simulink, permettant d'utiliser ROS depuis Matlab/Simulink.

Dans l'architecture ROS, chaque programme autonome est appelé *noeud*, et un ensemble de *noeud* est appelé *package*. La communication entre les noeuds se fait par l'échange de messages via des canaux appelés *topics*, selon un modèle *publisher-subscriber*. Un noeud *publisher* envoie des données sur un topic, tandis qu'un noeud *subscriber* les récupère. Lorsqu'ils sont connectés au même réseau, plusieurs appareils peuvent accéder aux mêmes topics. Cela facilite grandement la communication entre les différents noeuds, qu'ils soient sur le même appareil ou sur des appareils différents.

6.1.2 Système Optitrack - Logiciel Motive [7]

Optitrack est un système de motion capture (capture de mouvement) qui permet d'enregistrer et d'analyser les mouvements d'objets dans l'espace en temps réel à partir des données d'un ensemble de caméras infrarouges.

Motive, le logiciel propriétaire développé par OptiTrack, exploite ces données pour suivre des objets auxquels sont attachés des marqueurs réfléchissants et déterminer leur position et orientation dans l'espace. L'un des avantages de Motive est sa compatibilité avec le protocole Virtual-Reality Peripheral Network (VRPN) (Virtual-Reality Peripheral Network), qui permet de transmettre en temps réel les données des objets vers d'autres applications (ici, ROS2, via un noeud Python). Ainsi, un ensemble de bille réfléchissantes sont placées sur les robots, permettant de récupérer leur position et orientation en temps réel.

6.1.3 Turtlebot Mecanum

Comme mentionné précédemment, avant d'implémenter le programme de contrôle événementiel sur des drones, celui-ci sera dans un premier temps développé et testé sur des robots mobiles de type Turtlebot, équipés de roues Mecanum (Figure 6.1). Les roues Mecanum sont des roues particulières, composées de plusieurs galets montés en biais autour de leur circonférence, ce qui permet au robot de réaliser des mouvements dans toutes les directions du plan en combinant les vitesses de rotation de chaque roue. Ce type de robot est dit *holonome*. Dans le cadre du projet eSWARM, ces robots servent de substituts aux drones afin de simplifier les expérimentations. Ce choix permet de passer d'un système en trois dimensions à un système bidimensionnel, tout en évitant les risques de collisions ou de chutes inhérents à l'utilisation de drones.

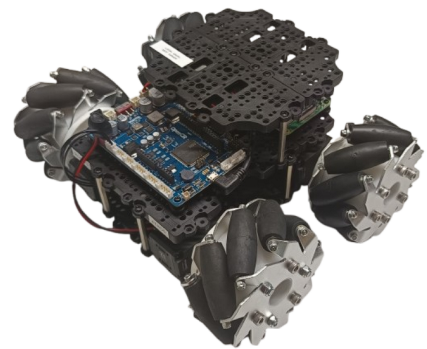


FIGURE 6.1 – TurtleBot Mecanum

Chaque robot est composé de quatre moteurs Dynamixel, d'une carte OpenCR assurant le pilotage bas niveau, et d'une Raspberry Pi 4 servant d'unité de calcul principale. Ils ont été conçus et programmés initialement sous ROS1 dans le cadre d'un projet de 5ème année à l'INSA Strasbourg. Trois robots étaient disponibles au début du stage, portant les noms des trois mousquetaires : Athos, Porthos et Aramis [8]. Trois robots supplémentaires ont été construits et programmés durant le stage : D'Artagnan, Edmon et Mercedes. Les six robots ont été programmés en ROS2 (annexe ??).

La cinématique de ce type de robot est donnée dans [9].

6.1.4 Logiciels utilisés

Les principaux logiciels utilisés sont :

- **Visual Studio Code (VS Code)**, qui est un Integrated Development Environment (IDE) (environnement de développement intégré) permettant de développer en Python, C++ et d'autres langages. Il est utilisé pour écrire le code des algorithmes de contrôle en Python. VS Code dispose de nombreuses extensions, dont une extension *ssh* permettant de se connecter à distance à la Raspberry Pi des robots. Cela permet d'avoir accès aux fichiers de cette dernière via l'explorateur de fichiers de l'IDE, et de modifier les fichiers directement depuis l'IDE, facilitant grandement le développement.
- **Matlab/Simulink**, qui est un environnement de développement pour la simulation et la modélisation de systèmes dynamiques. Il a surtout été utilisé pour visualiser les données en temps réel, et tracer les différents graphiques donnés en partie 7.
- **IDE Arduino**, afin de programmer la carte OpenCR des robots.
- **GitHub**, qui est une plateforme en ligne permettant d'héberger du code, de le versionner et de collaborer à plusieurs sur un même projet. GitHub est particulièrement utile pour les systèmes distribués, car il suffit d'utiliser la commande *git clone* sur chaque robot pour récupérer la dernière version du code, ce qui simplifie grandement la synchronisation du projet sur plusieurs machines.

Les programmes réalisés durant le stage sont disponibles sur le Github suivant :

<https://github.com/watson67/mecanum>

Différents fichiers *README* s'y trouvent pour des explications plus détaillées.

6.2 Essaim de robots

6.3 Organisation du système avec Robot Operating System (ROS)

7 Présentations des résultats

Dans cette partie, les résultats des différentes expérimentations sont présentés. Ils sont organisés en plusieurs sections : la première concerne l'application simple de l'algorithme de [5] aux robots Mecanum, la seconde porte sur le contrôle événementiel, et la troisième sur une commande prédictive.

La méthode de test est identique dans chaque section, c'est-à-dire que la formation et la trajectoire à parcourir sont les mêmes dans chaque cas. La formation adoptée est une formation carrée, illustrée par le graphe ci-contre (Figure 7.1). Deux robots sont considérés comme voisins s'ils partagent un côté du carré (les robots situés en diagonale ne sont donc pas voisins). Bien que six robots soient disponibles, seuls quatre sont utilisés pour les tests, afin de simplifier la visualisation des résultats. Des essais avec six robots sont donnés en annexe.

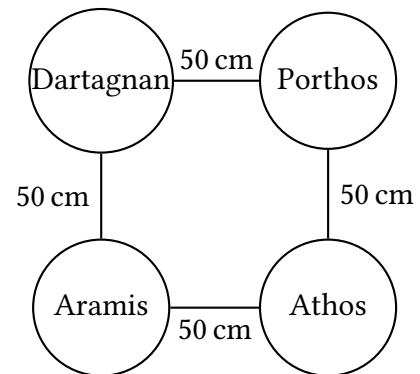


FIGURE 7.1 – Graphe de la formation

7.1 Algorithme classique

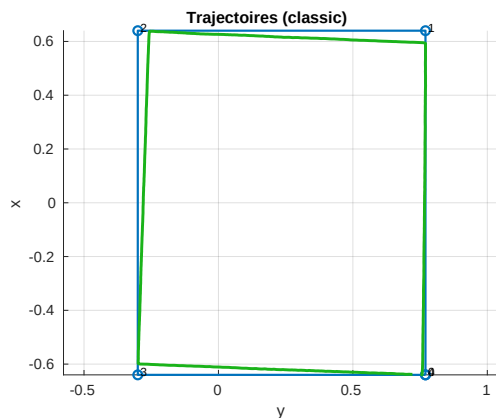


FIGURE 7.2 – Trajectoire du barycentre de l'essaim

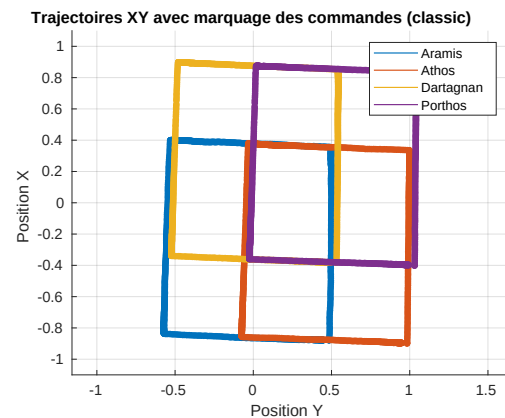


FIGURE 7.3 – Trajectoire de chaque robot de l'essaim

Note : Dans l'algorithme, il est considéré que l'essaim ait atteint sa destination si il est à 5cm du point-cible, expliquant ainsi les légères différences avec la trajectoire réelle tracée en bleue sur la Figure 7.2.

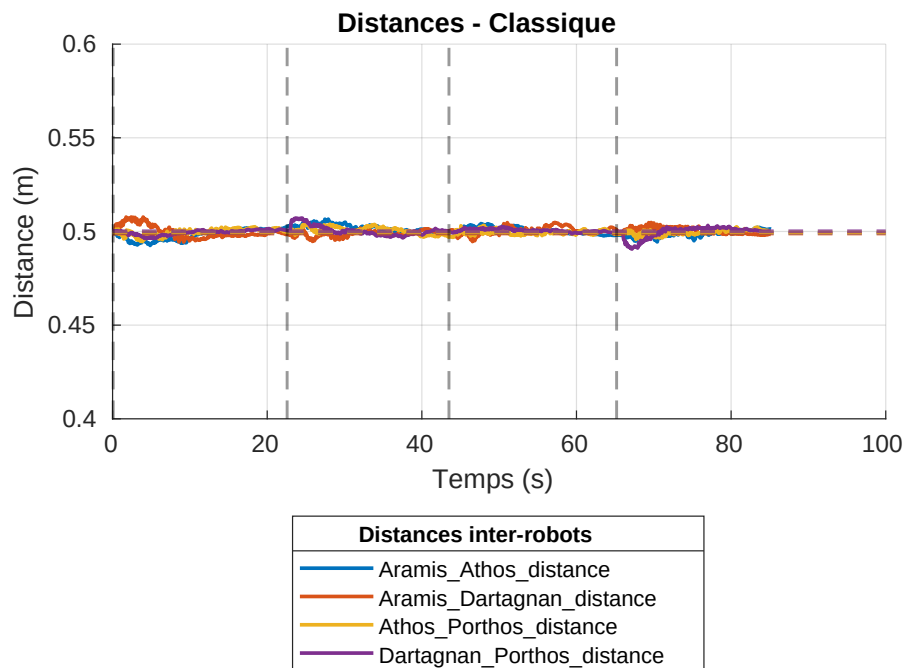


FIGURE 7.4 – Evolution des inter-distances entre les robots

Les résultats obtenus sont très satisfaisants, la formation et les interdistances sont maintenus tout au long du parcours.

Il est possible de tester la robustesse de l'algorithme en créant manuellement une perturbation. Pour cela, un robot (ici Athos) est déplacé à la main durant l'expérience de sorte à créer une erreur dans les inter-distances à maintenir.

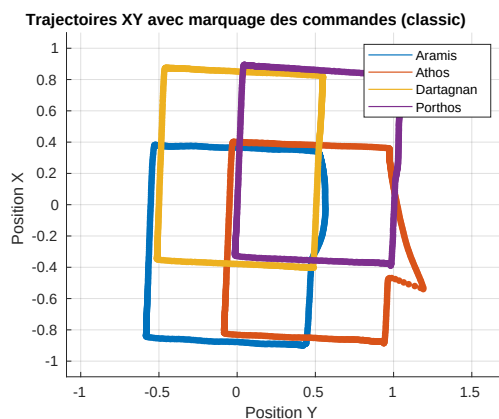


FIGURE 7.5 – Trajectoire des robots avec une perturbation

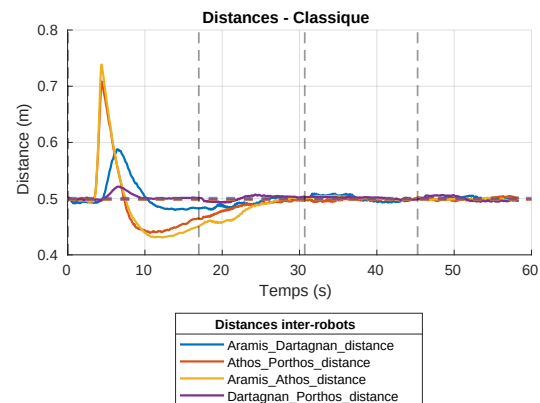


FIGURE 7.6 – Evolution des inter-distances entre les robots avec perturbation

L'algorithme met environ 30 secondes pour corriger la perturbation, ce qui est relativement long. Cependant, il est important de noter que l'algorithme parvient à rétablir la formation et les inter-distances initiales, ce qui est un point positif.

7.2 Commande événementielle

7.2.1 Commande événementielle classique

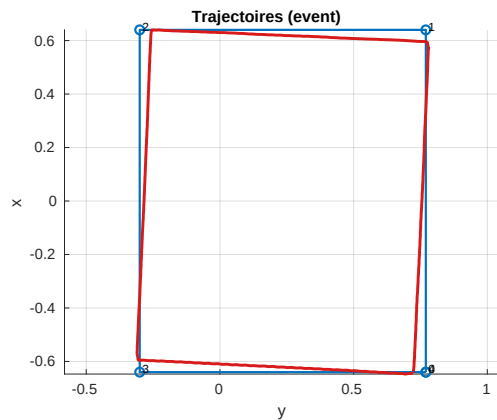


FIGURE 7.7 – Trajectoire du barycentre de l'essaim

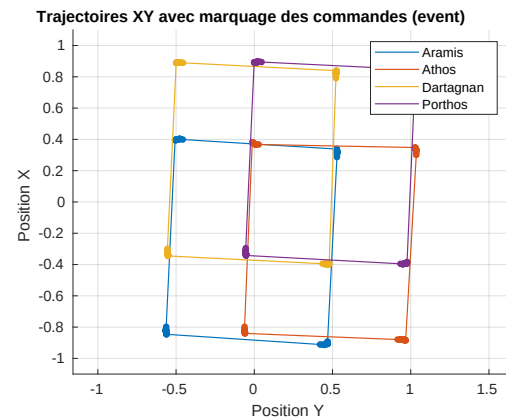


FIGURE 7.8 – Trajectoire de chaque robot de l'essaim

La Figure 7.8 illustre les trajectoires de chaque robot de l'essaim, ainsi que les points où les commandes sont calculées et appliquées, correspondant aux instants de déclenchement des événements. Il faut noter que les événements sont surtout déclenchés lorsque l'on se rapproche du point-cible (chaque coin de carré), ce qui s'explique simplement par le fait qu'à ces moments, les robots doivent ralentir pour ne pas dépasser le point cible.

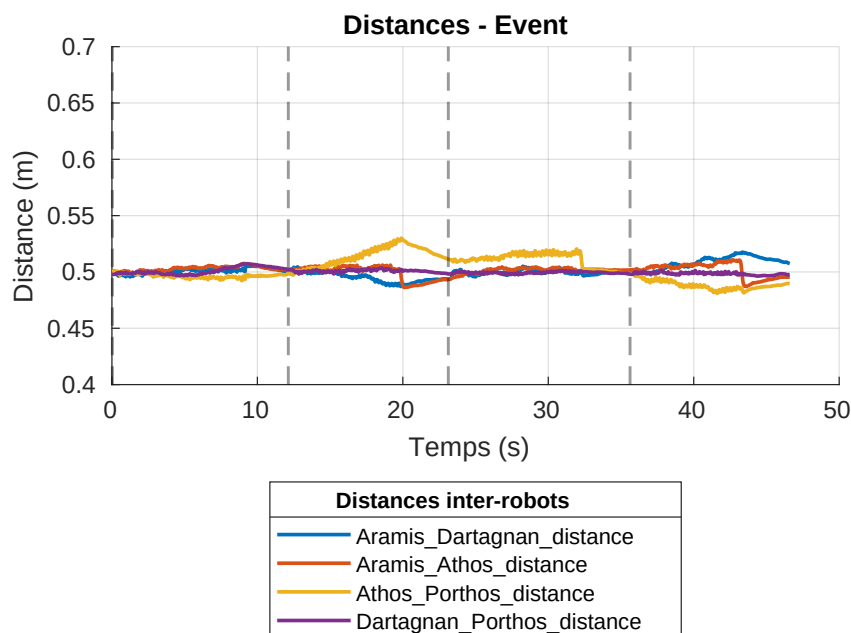


FIGURE 7.9 – Evolution des inter-distances entre les robots

Les résultats obtenus sont très satisfaisants, la formation et les interdistances sont maintenues tout au long du parcours. Cependant, une information non-visible sur les figures ci-dessus est la vitesse de convergence. Les robots accélèrent et freinent très brusquement, sans transition pour adoucir leur mouvement avant ou après un événement. Cela peut entraîner des comportements peu naturels ou des à-coups dans la dynamique de l'essaim. Pour visualiser cela, il suffit de tracer les vitesses des robots au cours du temps, comme illustré dans la Figure.

RAJOUTER UNE COURBE POUR MONTRER LES VITESSES La robustesse de l'algorithme est ensuite à nouveau testée, en créant manuellement une perturbation en retenant Athos.

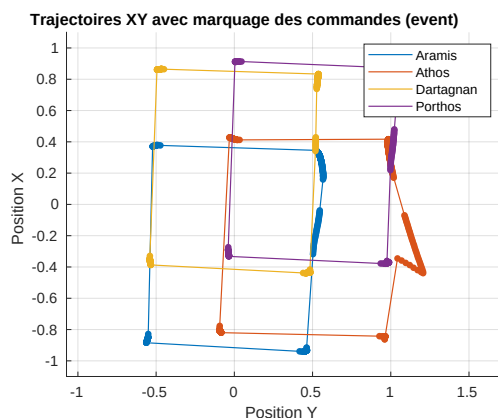


FIGURE 7.10 – Trajectoire des robots avec une perturbation

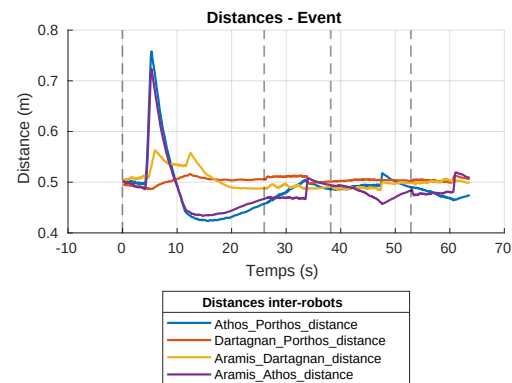


FIGURE 7.11 – Evolution des inter-distances entre les robots avec perturbation

Lors de la perturbation, la condition de déclenchement sur les écarts de distance est atteinte, ce qui entraîne le recalcul des commandes pour certains robots. Comme les commandes ne sont recalculées que lorsque l'écart dépasse ± 5 cm, il n'y a pas assez de corrections/calculs de commande pour permettre une convergence précise vers la valeur souhaitée.

7.2.2 Commande événementielle prédictive

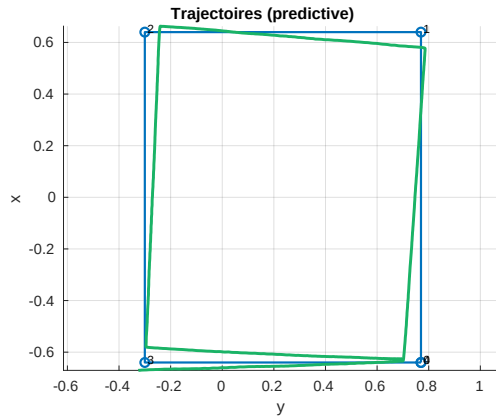


FIGURE 7.12 – Trajectoire du barycentre de l'essaim

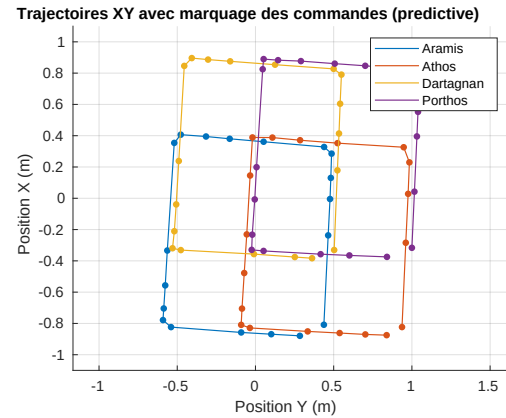


FIGURE 7.13 – Trajectoire de chaque robot de l'essaim

La Figure 7.13 illustre les trajectoires de chaque robot de l'essaim, ainsi que les positions publiées.

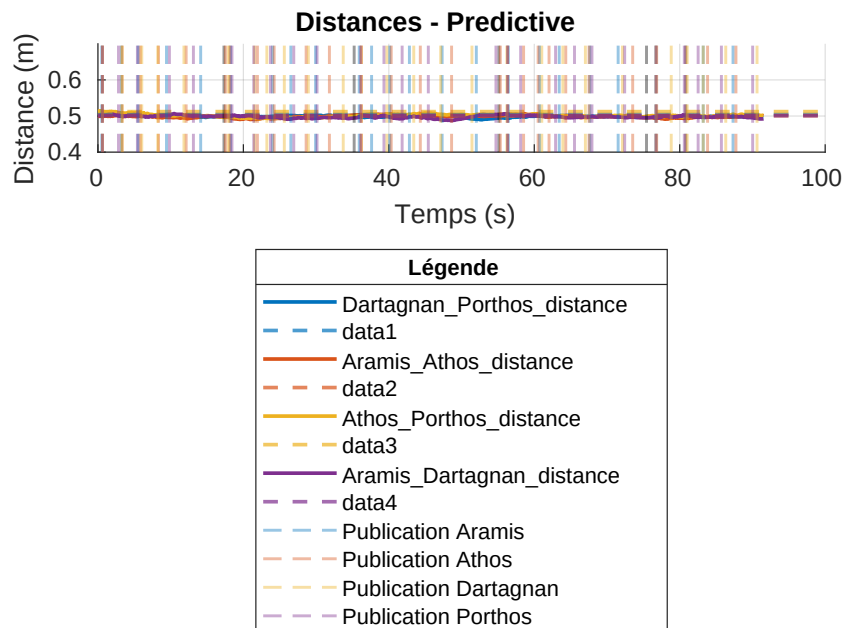


FIGURE 7.14 – Evolution des inter-distances entre les robots

Les résultats obtenus sont très satisfaisants, la formation et les interdistances sont maintenues tout au long du parcours.

La robustesse de l'algorithme est ensuite à nouveau testée, en créant manuellement une perturbation en retenant Athos.

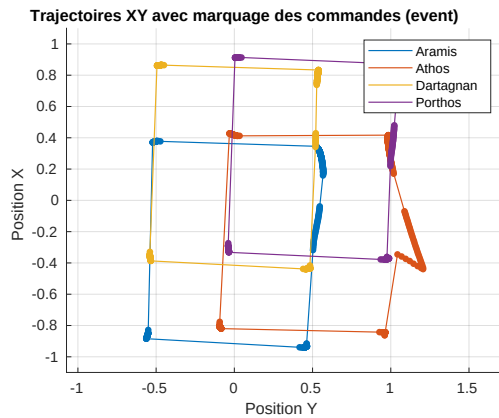


FIGURE 7.15 – Trajectoire des robots avec une perturbation

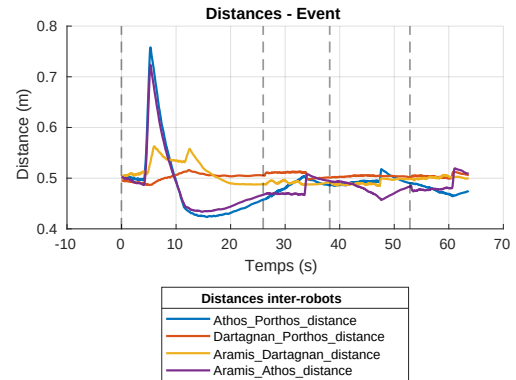


FIGURE 7.16 – Evolution des inter-distances entre les robots avec perturbation

Lors de la perturbation, la condition de déclenchement sur les écarts de distance est atteinte, ce qui entraîne le recalcul des commandes pour certains robots. Comme les commandes ne sont recalculées que lorsque l'écart dépasse ± 5 cm, il n'y a pas assez de corrections/calculs de commande pour permettre une convergence précise vers la valeur souhaitée.

7.3 Comparaisons des différentes méthodes

8 Conclusion

Bibliographie

- [1] Augustin BONNEL. *Modeling and control of a modular system of UAVs*. Rapport de PFE. Strasbourg : INSA, 2024, p. 60.
- [2] Jungwon SEO, Jamie PAIK et Mark YIM. « Modular Reconfigurable Robotics ». In : *Annual Review of Control, Robotics, and Autonomous Systems* 2 (Volume 2, 2019 3 mai 2019). Publisher : Annual Reviews, p. 63-88. ISSN : 2573-5144. DOI : 10.1146/annurev-control-053018-023834. URL : <https://www.annualreviews.org/content/journals/10.1146/annurev-control-053018-023834> (visité le 10/03/2025).
- [3] Sam BROOKS et Rajkumar ROY. « An overview of self-engineering systems ». In : *Journal of Engineering Design* 32.8 (3 août 2021). Publisher : Taylor & Francis _eprint, p. 397-447. ISSN : 0954-4828. DOI : 10.1080/09544828.2021.1914323. URL : <https://doi.org/10.1080/09544828.2021.1914323> (visité le 10/03/2025).
- [4] David SALDAÑA et al. « ModQuad : The Flying Modular Structure that Self-Assembles in Mid-air ». In : *2018 IEEE International Conference on Robotics and Automation (ICRA)*. 2018 IEEE International Conference on Robotics and Automation (ICRA). ISSN : 2577-087X. Mai 2018, p. 691-698. DOI : 10.1109/ICRA.2018.8461014. URL : <https://ieeexplore.ieee.org/document/8461014/?arnumber=8461014> (visité le 04/03/2025).
- [5] Daravuth KOUNG et al. « Consensus-based formation control and obstacle avoidance for non-holonomic multi-robot system ». In : *2020 16th International Conference on Control, Automation, Robotics and Vision (ICARCV)*. 2020 16th International Conference on Control, Automation, Robotics and Vision (ICARCV). Déc. 2020, p. 92-97. DOI : 10.1109/ICARCV50220.2020.9305426. URL : <https://ieeexplore.ieee.org/abstract/document/9305426> (visité le 09/04/2025).
- [6] *ROS 2 Documentation — ROS 2 Documentation : Humble documentation*. URL : <https://docs.ros.org/en/humble/index.html> (visité le 22/04/2025).
- [7] *Motive - In Depth*. OptiTrack. URL : <http://optitrack.com/software/motive/index.html> (visité le 22/04/2025).
- [8] Aude VRIGNAUD et Théo OLIVEIRA. *Swarm of holonomous robots*. Rapport de projet. INSA Strasbourg, 2025, p. 30.
- [9] Xiangquan TAN, Shuliang ZHANG et Qingwen WU. « Research on Omnidirectional Indoor Mobile Robot System Based on Multi-sensor Fusion ». In : *2021 5th International Conference on Vision, Image and Signal Processing (ICVISIP)*. 2021 5th International Conference on Vision, Image and Signal Processing (ICVISIP). Déc. 2021, p. 111-117. DOI : 10.1109/ICVISIP54630.2021.00028. URL : <https://ieeexplore.ieee.org/document/9700878> (visité le 07/04/2025).
- [10] Daravuth KOUNG. « Cooperative navigation of a fleet of mobile robots ». Thèse de doct. École centrale de Nantes, 19 oct. 2022. URL : <https://theses.hal.science/tel-04025814> (visité le 06/06/2025).
- [11] Wei REN et Randal W. BEARD. *Distributed Consensus in Multi-vehicle Cooperative Control*. Réd. par E. D. SONTAG et al. Communications and Control Engineering. London : Springer London, 2008. ISBN : 978-1-84800-014-8 978-1-84800-015-5. DOI : 10.1007/978-1-84800-015-5. URL : <http://link.springer.com/10.1007/978-1-84800-015-5> (visité le 06/06/2025).

- [12] *Graph theory*. In : *Wikipedia*. Page Version ID : 1289635162. 9 mai 2025. URL : https://en.wikipedia.org/w/index.php?title=Graph_theory&oldid=1289635162 (visité le 06/06/2025).
- [13] R. OLFATI-SABER. « Flocking for multi-agent dynamic systems : algorithms and theory ». In : *IEEE Transactions on Automatic Control* 51.3 (mars 2006), p. 401-420. ISSN : 1558-2523. DOI : 10.1109/TAC.2005.864190. URL : <https://ieeexplore.ieee.org/document/1605401> (visité le 06/06/2025).
- [14] Osamah SAIF, Isabelle FANTONI et Arturo ZAVALA-RÍO. « Distributed integral control of multiple UAVs : precise flocking and navigation ». In : *IET Control Theory & Applications* 13.13 (3 sept. 2019). Publisher : The Institution of Engineering and Technology, p. 2008-2017. DOI : 10.1049/iet-cta.2018.5684. URL : <https://digital-library.theiet.org/doi/10.1049/iet-cta.2018.5684> (visité le 06/06/2025).

Annexes

A Théorie des graphes

Cette partie s'inspire des documents [10, 11, 12].

A.1 Introduction

La théorie des graphes est un domaine des mathématiques et de l'informatique qui s'intéresse à l'étude des graphes, des structures composées de nœuds (ou sommets) pouvant être connectés entre eux par des arêtes. Ces graphes permettent de modéliser des relations ou des interactions entre objets. Un exemple de graphe est illustré en figure A.1. Dans le cadre des essaims de robots, cette représentation est couramment utilisée : chaque robot est associé à un nœud, et une arête peut symboliser une interaction ou une proximité, par exemple avec un robot voisin. Ainsi, la théorie des graphes permet de représenter la topologie du système.

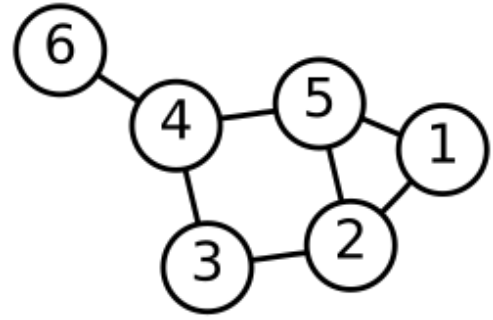


FIGURE A.1 – Exemple de graphe

A.2 Définition mathématique

A.2.1 Principe

Comme expliqué précédemment en introduction, un graphe $\mathcal{G}$ est composé d'un ensemble de nœuds, noté $\mathcal{E}$, et d'un ensemble d'arêtes, noté $\mathcal{V}$.

- $\mathcal{V} = \{1, 2, \dots, n\}$, l'ensemble des nœuds.
- $\mathcal{E} \subseteq \{(i, j) : i, j \in \mathcal{V}, j \neq i\}$. Une arête est composée de nœuds i et j

Un graphe peut être représenté sous la forme d'une matrice d'adjacence $\mathcal{A}(\mathcal{G})$ de taille $n \times n$ (où n est le nombre de nœuds), dont les termes $a_{i,j}$ indiquent la présence ou non d'une arête entre les nœuds i et j . Chaque élément $a_{i,j}$ de la matrice est défini par :

$$a_{i,j} = \begin{cases} 1 & \text{si } (i, j) \in \mathcal{E} \text{ c'est-à-dire une qu'il existe une arête entre les nœuds } i \text{ et } j \\ 0 & \text{sinon.} \end{cases}$$

Par exemple, sur la Figure A.1, il y a 6 nœuds formant les arêtes suivantes :

- le nœud 1 est lié au nœud 2 et au nœud 5
- le nœud 2 est lié au nœud 3 et au nœud 5
- le nœud 3 est lié au nœud 4
- le nœud 4 est lié au nœud 5 et au nœud 6

Il est alors possible d'en déduire la matrice d'adjacence suivante :

$$\mathcal{A}(\mathcal{G}) = \begin{bmatrix} 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

La matrice obtenue est symétrique, car le graphe est non orienté (voir Partie A.2.2) ce qui signifie que les connexions sont valables dans les deux sens, et la diagonale est nulle car il n'y a pas, dans ce cas, de connexion entre un nœud et lui-même.

A.2.2 Propriétés

A.2.2.1 Graphes orientés

Les arêtes peuvent avoir une orientation ; le graphe est alors dit **orienté**. Dans ce type de graphe, chaque arête possède une direction spécifique qui détermine le sens de la relation entre les nœuds. Un exemple de graphe orienté est donné ci-contre.

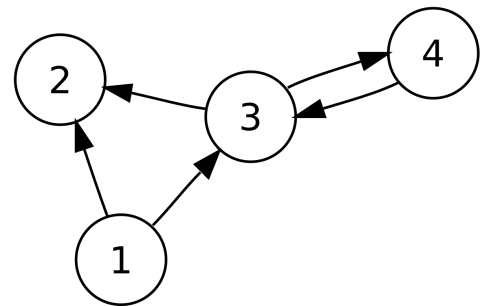


FIGURE A.2 – Graphe orienté

Dans ce cas, la matrice d'adjacence est la suivante :

$$\mathcal{A}(\mathcal{G}) = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{bmatrix}$$

Si le graphe est orienté, la matrice d'adjacence n'est plus symétrique, sauf si toutes les arêtes sont bidirectionnelles, ce qui revient à considérer le graphe comme étant non-orienté.

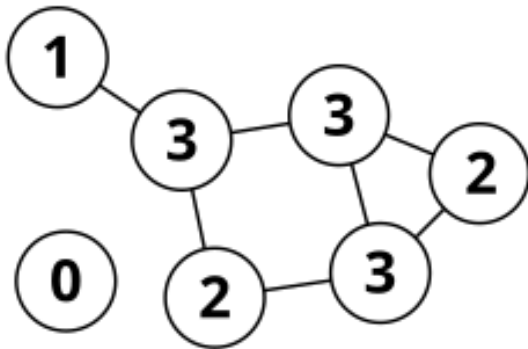
A.2.2.2 Connexité

Un graphe est dit **connexe** s'il existe un chemin entre n'importe quel couple de nœuds (i, j) du graphe $\mathcal{G}$. Autrement dit, pour toute paire de sommets, il doit exister une chaîne ou un chemin permettant de relier ces deux nœuds en passant par les arêtes du graphe.

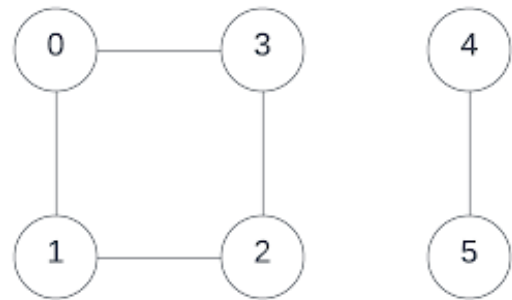
Pour illustrer cette notion, les graphes présentés ci-dessous constituent des exemples de graphes non connexes.

Sur la Figure A.3A.3.1, le nœud 0 n'est pas relié à d'autres nœuds du graphe, il n'existe donc pas de chemin permettant de relier le nœud 0 et le nœud 1 par exemple. Sur la Figure A.3A.3.2, il n'existe par exemple pas de chemin reliant le nœud 4 et le nœud 1, car ces nœuds appartiennent à des composantes connexes distinctes. Lorsqu'un graphe est orienté, plusieurs niveaux de connexité peuvent être distingués :

- **Connexité faible** : le graphe est connexe lorsque l'orientation des arêtes est ignorée.



(A.3.1) Exemple 1 de graphe non connexe



(A.3.2) Exemple 2 de graphe non connexe

FIGURE A.3 – Deux exemples de graphes non connexes

- **Connexité unilatérale** : pour chaque paire de sommets (i, j) , un chemin orienté existe de i vers j ou de j vers i , mais pas nécessairement dans les deux sens.
- **Connexité forte** : un chemin orienté relie chaque sommet i à tout autre sommet j , peu importe le sens.

A.2.2.3 Arbre couvrant (Spanning Tree)

Un **arbre couvrant** est une sous-structure d'un graphe connexe qui relie tous les nœuds du graphe avec un minimum d'arêtes, sans former de cycle. Un graphe de n nœuds possède un arbre couvrant composé de exactement $n - 1$ arêtes. Un graphe admet un arbre couvrant *si et seulement si* il est connexe.

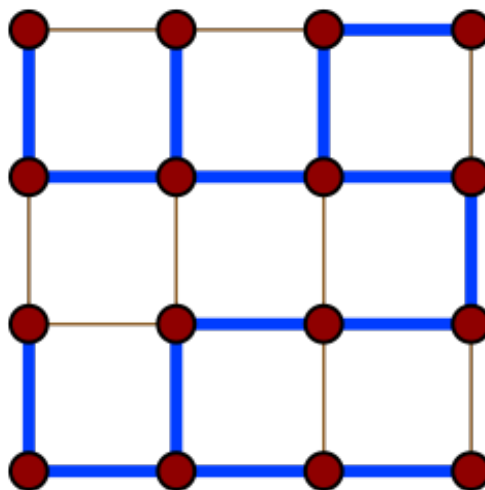


FIGURE A.4 – Exemple d'un arbre couvrant (en bleu)

A.2.2.4 Rigidité

Un graphe est rigide si les distances (taille des arête) entre certains couples de nœuds suffisent à garantir que la structure globale de la formation ne peut pas être modifiée (par exemple être déformée ou pliée) sans altérer ces distances. En d'autres termes, la formation est préservée, à une translation

ou une rotation près.

Plus formellement, un graphe est dit **rigide en dimension 2** si, pour une configuration générique des positions des nœuds dans le plan, la seule manière de déplacer les nœuds tout en conservant les longueurs des arêtes est de faire une translation ou une rotation globale de l'ensemble du graphe.

Pour qu'un graphe non orienté à n nœuds soit rigide dans le plan, une condition nécessaire est qu'il possède au moins $2n - 3$ arêtes. Cependant, ce n'est pas suffisant : la répartition de ces arêtes est également importante.

Dans les applications robotiques, la rigidité garantit que les robots peuvent estimer leur position relative les uns par rapport aux autres à partir de mesures de distance locales, ce qui est essentiel pour le maintien de formations.

A.3 Lien avec un essaim de robot

Il faut choisir avec soin le type de graphe à utiliser lors de la conception d'un essaim. Dans le cadre du projet eSWARM, l'objectif est de mettre en place un système d'essaim décentralisé, ce qui implique la non orientation ainsi que la connexité du graphe. En effet, chaque agent de l'essaim doit avoir accès aux informations des autres agents de l'essaim (par exemple, les distances inter-agents ou la position du robot etc).

B Contrôle de l'essaim

Cette partie reprend les travaux [10].

Soit un ensemble de n robots noté $\mathcal{V}$.

B.1 Introduction

Soit un robot i ; ce dernier possède au sein de l'essaim, un ensemble $\mathcal{N}_i$ de voisins, avec qui il forme des arêtes.

$$\mathcal{N}_i = \{j \in \mathcal{V} : (i, j) \in \mathcal{E}\} = \{j \in \mathcal{V} : \|\mathbf{p}_i - \mathbf{p}_j\| < c\} \quad (\text{B.1})$$

où $c > 0$ est la portée d'interaction entre robots, et $\|\cdot\|$ dénote la norme euclidienne. $\mathbf{p}_i$ et $\mathbf{p}_j$ sont les coordonnées cartésiennes des robots i et j respectivement. Ainsi, deux robots sont considérés comme voisins s'ils peuvent communiquer directement ou s'ils se trouvent à une distance inférieure au seuil c .

Aussi, pour conserver la formation, le robot i doit maintenir une distance désirée d_{ij} avec chacun de ses voisins j . Mathématiquement, cela s'écrit :

$$\|\mathbf{p}_i - \mathbf{p}_j\| < d_{ij} \quad \forall j \in \mathbf{N}_i \quad (\text{B.2})$$

B.2 Définition du consensus

Cette partie s'inspire de [11].

La cinématique à intégrateur simple est le modèle dynamique le plus basique, dans lequel la dérivée de la position (vitesse) est directement égale à l'entrée de commande, sans inertie ni frottement. Soit le système de n robots, dont la dynamique est :

$$\dot{\xi}_i = u_i, \quad i = 1, \dots, n \quad (\text{B.3})$$

où $\xi_i \in \mathbb{R}^m$ représente l'état (position) de l'agent i et $u_i \in \mathbb{R}^m$ est l'entrée de commande (vitesse) appliquée à cet agent. Le **consensus** désigne l'accord progressif des différents agents du système sur une certaine variable d'intérêt, par exemple la vitesse dans un essaim de robots mobiles.

Pour atteindre le consensus, chaque agent utilise une loi de commande basée sur les écarts avec ses voisins. L'algorithme de consensus distribué s'écrit :

$$u_i = - \sum_{j \in \mathcal{N}_i(t)} (\xi_i - \xi_j) \quad (\text{B.4})$$

Cette commande fait que chaque agent i se déplace vers la position moyenne de ses voisins, en appliquant une force proportionnelle à l'écart de position. L'ensemble $\mathcal{N}_i(t)$ dénote les voisins variant dans le temps du robot i .

Le consensus est atteint si, pour toutes conditions initiales $\xi_i(0)$, les états de tous les agents convergent vers une valeur commune :

$$\xi_i(t) \rightarrow \xi_j(t) \quad \text{quand } t \rightarrow \infty, \quad \forall i, j$$

Pour que le consensus ait lieu, il faut que le graphe du système ait un **arbre couvrant** (voir partie A.2.2.3).

B.3 Application à un essaim

Cette partie s'inspire largement de [10, 13, 14].

Soit la matrice d'adjacence spatiale $\mathcal{A}(p)$, tel que :

$$a_{i,j} = \begin{cases} 0 & \text{si } i = j \\ \frac{\rho_h(\|\mathbf{p}_i - \mathbf{p}_j\|_\sigma)}{\|c\|_\sigma} & \text{sinon} \end{cases} \quad (\text{B.5})$$

où ρ_h est appelée fonction de transition et $\|\cdot\|_\sigma$ est appelée norme sigma. La fonction de transition est une fonction scalaire variant entre 0 et 1 : $\mathbb{R}^+ \rightarrow [0, 1]$. Avec $h \in (0, 1)$, cette fonction est définie comme suit :

$$\rho_h(z) = \begin{cases} 1, & z \in [0, h) \\ \frac{1}{2} \left[1 + \cos \left(\pi \frac{(z-h)}{(1-h)} \right) \right], & z \in [h, 1] \\ 0, & \text{sinon} \end{cases} \quad (2.14)$$

La norme sigma est une application d'un vecteur vers une valeur positive : $\mathbb{R}^n \rightarrow \mathbb{R}^+$, $n \in \mathbb{Z}^+$. Cette norme est définie comme suit :

$$\|z\|_\sigma = \frac{1}{\epsilon} \left[\sqrt{1 + \epsilon \|z\|^2} - 1 \right] \quad (2.15)$$

avec $\epsilon > 0$, et son gradient peut être exprimé comme :

$$\sigma_\epsilon(z) = \nabla \|z\|_\sigma = \frac{z}{\sqrt{1 + \epsilon \|z\|^2}} = \frac{z}{1 + \epsilon \|z\|_\sigma} \quad (2.16)$$

$\|z\|_\sigma$ est différentiable partout, contrairement à la norme euclidienne classique. Cela signifie qu'elle reste continue et dérivable en tout point, et permet ainsi d'utiliser des outils d'analyse et de commande fondés sur les gradients ou jacobiens sans singularité. Globalement, cette norme permet d'éviter un comportement instable lorsque les robots sont très proches les uns des autres.

Une fonction de potentiel d'attraction/répulsion lisse par morceaux est définie comme :

$$\Psi_\alpha(z) = \int_{\|d\|_\sigma}^z \Phi_\alpha(\delta) d\delta \quad (\text{B.6})$$

Cette fonction permet d'intégrer d'une fonction Φ_α qui a un minimum à $z = \|d\|_\sigma$ et une coupure finie lorsque $z \geq \|c\|_\sigma$ (l'intégrale devient nulle au delà du seuil $\|c\|_\sigma$).

$$\Phi_\alpha(z) = \rho_h \left(\frac{z}{\|c\|_\alpha} \right) \Phi(z - \|d\|_\alpha) \quad (\text{B.7})$$

$$\Phi(z) = \frac{1}{2} [(a+b)\sigma_1(z+e) + (a-b)] \quad (\text{B.8})$$

où $\sigma_1(z) = \frac{z}{\sqrt{1+z^2}}$. Avec $0 < a \leq b$ et $e = \frac{|a-b|}{\sqrt{4ab}}$, la fonction sigmoïdale asymétrique $\Phi(z)$ est nulle lorsque $z = 0$.

C Modèle Mecanum

Les roues Mecanum sont des roues particulières, composées de plusieurs galets montés en biais autour de leur circonférence, ce qui permet au robot de réaliser des mouvements dans toutes les directions du plan en combinant les vitesses de rotation de chaque roue. Ce type de robot est dit *holonome*. Dans le cadre du projet eSWARM, ces robots servent de substituts aux drones afin de simplifier les expérimentations. Ce choix permet de passer d'un système en trois dimensions à un système bidimensionnel, tout en évitant les risques de collisions ou de chutes inhérents à l'utilisation de drones.

Chaque robot est composé de quatre moteurs Dynamixel, d'une carte OpenCR assurant le pilotage bas niveau, et d'une Raspberry Pi 4 servant d'unité de calcul principale. Ils ont été conçus et programmés initialement sous Robot Operating System version 1 (ROS1) dans le cadre d'un projet de 5ème année à l'INSA Strasbourg. Trois robots sont disponibles, portant les noms des trois mousquetaires : Athos, Porthos et Aramis [8]. Avant de mettre en place un algorithme de contrôle, il a fallu migrer les robots vers Robot Operating System version 2 (ROS2).

La cinématique de ce type de robot est donnée dans [9]. Soit un robot mobile équipé de quatre roues Mecanum. Chaque roue est numérotée de 1 à 4, et sa vitesse angulaire est notée ω_i (pour $i = 1, 2, 3, 4$). La vitesse du centre de la roue est V_i .

Les paramètres utilisés sont :

- O : centre du robot
- R : rayon des roues.
- α : angle d'orientation des galets des roues (souvent 45°).
- a : demi-largeur du robot (distance entre le centre O et l'axe de la roue en y).
- b : demi-longueur du robot (distance entre O et la roue en x).
- V_X, V_Y : composantes de la vitesse linéaire du centre du robot selon les axes X et Y .
- ω : vitesse angulaire du robot autour de son centre O .

Le modèle cinématique inverse général est donné par

$$\begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \\ \omega_4 \end{bmatrix} = \frac{1}{R} \begin{bmatrix} 1 & -\frac{1}{\tan \alpha} & \frac{b+a \tan \alpha}{\tan \alpha} \\ 1 & \frac{1}{\tan \alpha} & \frac{b-a \tan \alpha}{\tan \alpha} \\ 1 & -\frac{1}{\tan \alpha} & \frac{b+a \tan \alpha}{\tan \alpha} \\ 1 & \frac{1}{\tan \alpha} & \frac{b-a \tan \alpha}{\tan \alpha} \end{bmatrix} \begin{bmatrix} V_X \\ V_Y \\ \omega \end{bmatrix}$$

Le modèle cinématique direct est :

$$\begin{bmatrix} V_X \\ V_Y \\ \omega \end{bmatrix} = \frac{R}{4} \begin{bmatrix} 1 & 1 & 1 & 1 \\ -1 & 1 & -1 & 1 \\ -\frac{1}{a+b} & -\frac{1}{a+b} & \frac{1}{a+b} & \frac{1}{a+b} \end{bmatrix} \begin{bmatrix} \omega_1 \\ \omega_2 \\ \omega_3 \\ \omega_4 \end{bmatrix}$$